WaveVis inverse-sheet architecture for Moana-ocean breaking-wave linkage arrays

Alan N. Pham

AO Labs, WaveVis simulator, inverse deformation sheet record

WaveVis is an inverse simulator for a mostly flat rectangular linkage sheet with one localized plunging-wave deformation rising out of the sheet. The primary visual target is the supplied breaking-wave reference family: Moana-like water stills plus the June 23, 2026 line drawing with a broad outer arch, inward spiral curl, smooth downturned tapered tip, smooth inner throat, and long low return. The intended artifact is not an extruded ribbon, half-pipe, swept strip, roof, bridge, or hollow shell. It is a full array of linkage cells over a fixed flat perimeter: the center region ramps upward, forms a rounded crest, projects a lip forward and downward, curls into the tapered lip tip, and fades smoothly back into the surrounding sheet in all directions. The reference geometry is an acceptance target, not a success claim. A rendered state is still open if the human-facing render reads as a tunnel, wall-covered cavity, bowl, dimple, angular blob, missing curl, hooked or abrupt tip, or broken linkage even when code checks pass. A verification URL is also part of the artifact: if the page ignores URL slider parameters, the screenshot is not evidence for the state Alan requested. This paper replaces the earlier narrow constraint-proof PDF as the visible system record. It defines the reference geometry, coordinate systems, curated breaking-wave target, visible slider semantics, topology invariants, rendering views, failure modes, verification gates, deployment state, and handoff procedure required before WaveVis work can be called correct.

Fresh-chat use. This PDF is intended to be the first artifact a new WaveVis chat reads. It is not only a public paper. It is the continuation manual for the current simulator: what shape is being made, what failures were rejected, which files own the implementation, how each slider is allowed to behave, what tests must pass, what public route is verified, what remains unresolved, and how to continue without re-learning the same mistakes from chat history.

Contents

1 Introduction

WaveVis exists to explore whether a cellular Sarrus-style linkage array can approximate visually meaningful three-dimensional sheet deformations while preserving mechanical readability. The simulator therefore has two simultaneous jobs. First, it must generate a target shape that resembles the supplied Moana ocean references rather than a generic height field. Second, it must keep the linkage graph legible: cells, nodes, connector pairs, and edges must remain visible and connected rather than being hidden under cosmetic surfaces or erased when the shape becomes difficult.

The current design problem is stricter than drawing a pretty profile. A side outline can look acceptable while the actual three-dimensional sheet has a wall across the underside, a cavity where the lip should be open, an extruded barrel, or zig-zagging linkage legs that no longer read as a mechanism. Conversely, a mesh can satisfy a local numerical check while failing the human outcome: a full rectangular sheet with a localized plunging wave. WaveVis must therefore treat the visible artifact, the mechanism topology, the parameter semantics, and the regression checks as one architecture.

The central correction captured here is that a breaking wave is not made by sweeping one two-dimensional profile sideways. A swept profile creates a tunnel, canopy, or half-pipe. The required object is a localized deformation field on a flat sheet:

$$\Phi : (x, y) \mapsto \big(x + \Delta_x(x, y),\ y + \Delta_y(x, y),\ \Delta_z(x, y)\big),$$

where the displacement is concentrated near the middle span, gradually fades toward the lateral sides, and is exactly zero on the perimeter. The curl must be strongest near the centerline and weaker toward the sides. This spatial falloff is part of the target shape, not a rendering trick.

1.1 Reference geometry

The design reference is the Moana ocean plus the June 23, 2026 supplied line drawing: a smooth, localized volume of water that rises out of a larger flat body, grows into a broad rounded crest, curls into a smaller inward spiral, and finishes with a smooth downturned tapered lip while leaving an open underside and a long low return. These reference frames are not decorative. They are the visual contract for the simulator's target geometry. A successful WaveVis state should read as a simplified linkage approximation of this water form: the outer radius is large and smooth, the inner curl radius is smaller, the lip projects forward before curling down, and the side field fades away instead of forming a

constant barrel. This is deliberately stricter than matching a side-profile curve: the full linkage array in side and isometric views must preserve the open curl without side walls, erratic zig-zags, disconnected cells, or hidden filler geometry.

Current verification boundary. This document records the target, architecture, and required gates. It does not by itself prove that the current generator has matched the supplied references. The rendered full-array artifact remains the source of truth. If the page, screenshots, or live route do not visibly match the Moana-like localized plunging wave while preserving linkage cells and flat perimeter, the correct state is “open visual mismatch,” not “reference matched.”

Figure 1. Primary supplied reference geometry. The target form is a localized plunging water sheet with a rounded crest, open underside, and forward/downward lip. The WaveVis sheet must approximate this geometry while preserving flat perimeter boundary nodes and visible linkage cells.

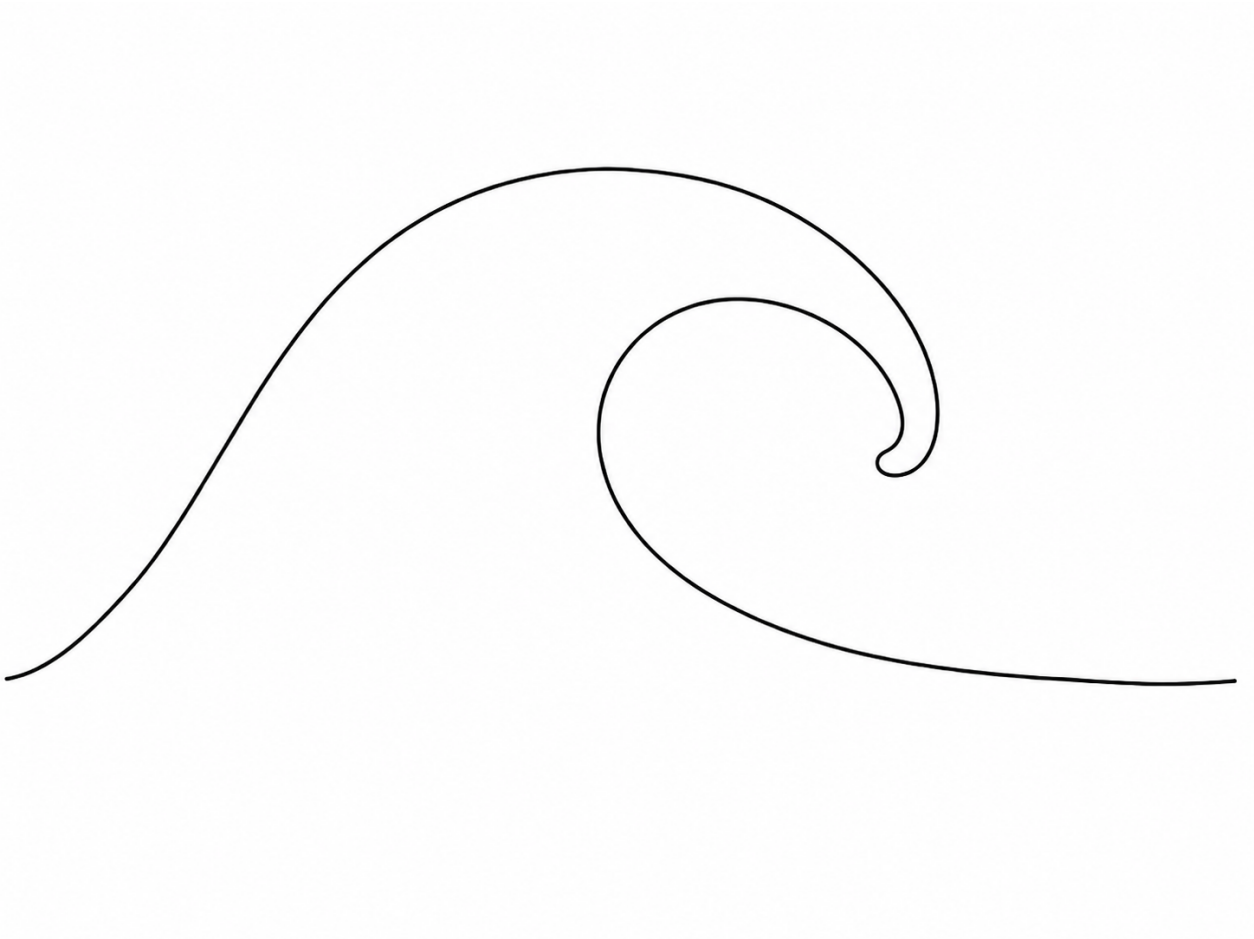

Figure 2. June 23 supplied side-profile reference. This image sharpens the visible target: broad outer arch, smaller inward curl, smooth downturned tapered tip, smooth curved inner throat, and long low return. It is a shape reference only; the three-dimensional WaveVis render must still preserve the full linkage sheet rather than becoming a two-dimensional trace or silhouette.

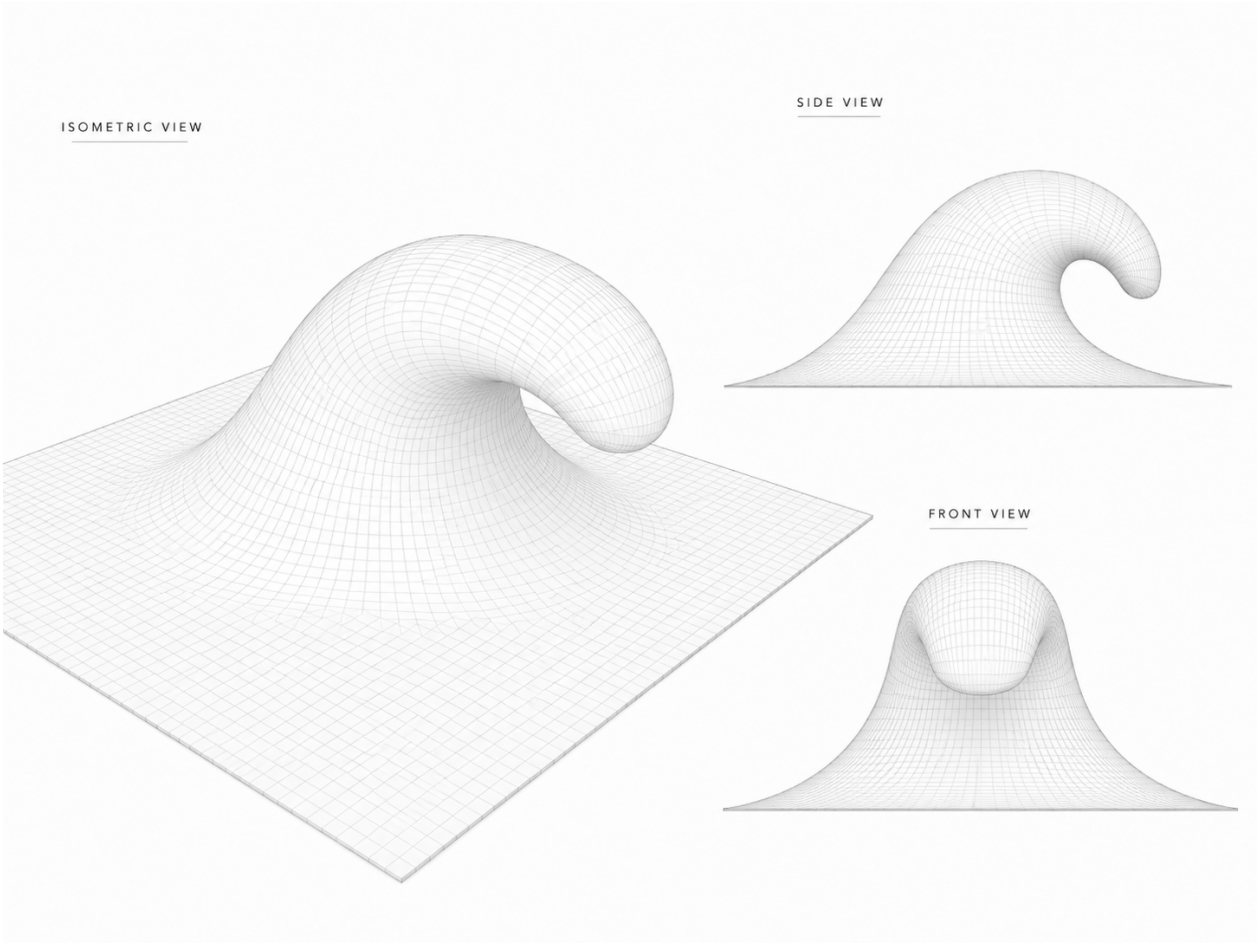

Figure 3. June 24 supplied smooth gridded reference overview. Alan re-supplied this image and said "remember." It is now a preserved project reference for the desired visual grammar: a continuous square sheet surface, rounded rising face, tube-like crest, downturned lip, open clean throat, and centered front view. A jagged linkage trace, side-wall tunnel, or metric-only improvement does not satisfy this reference.

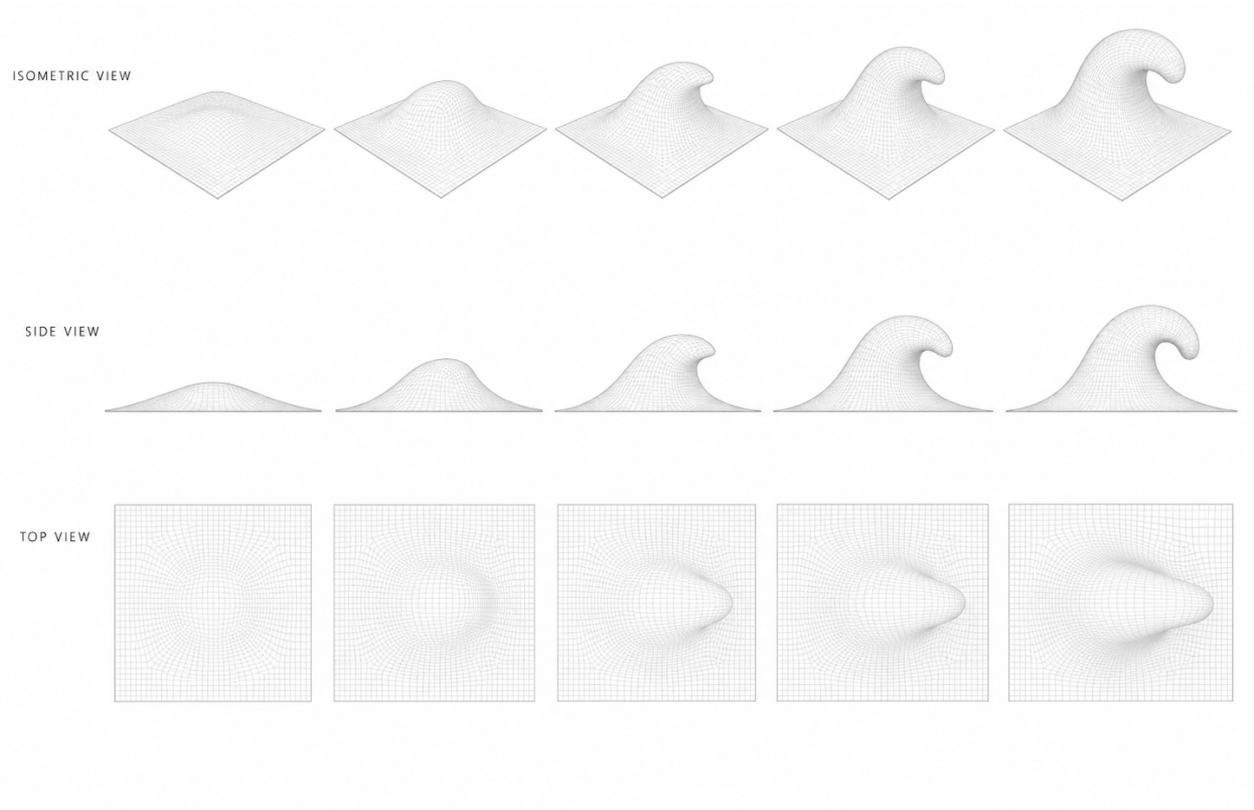

Figure 4. June 24 supplied smooth gridded reference sequence. The sequence records the intended progression across isometric, side, and top views: the deformation grows from a flat square sheet into a localized rounded wave with a curled lip while the top view remains a rounded localized body, not a triangular fan or swept strip.

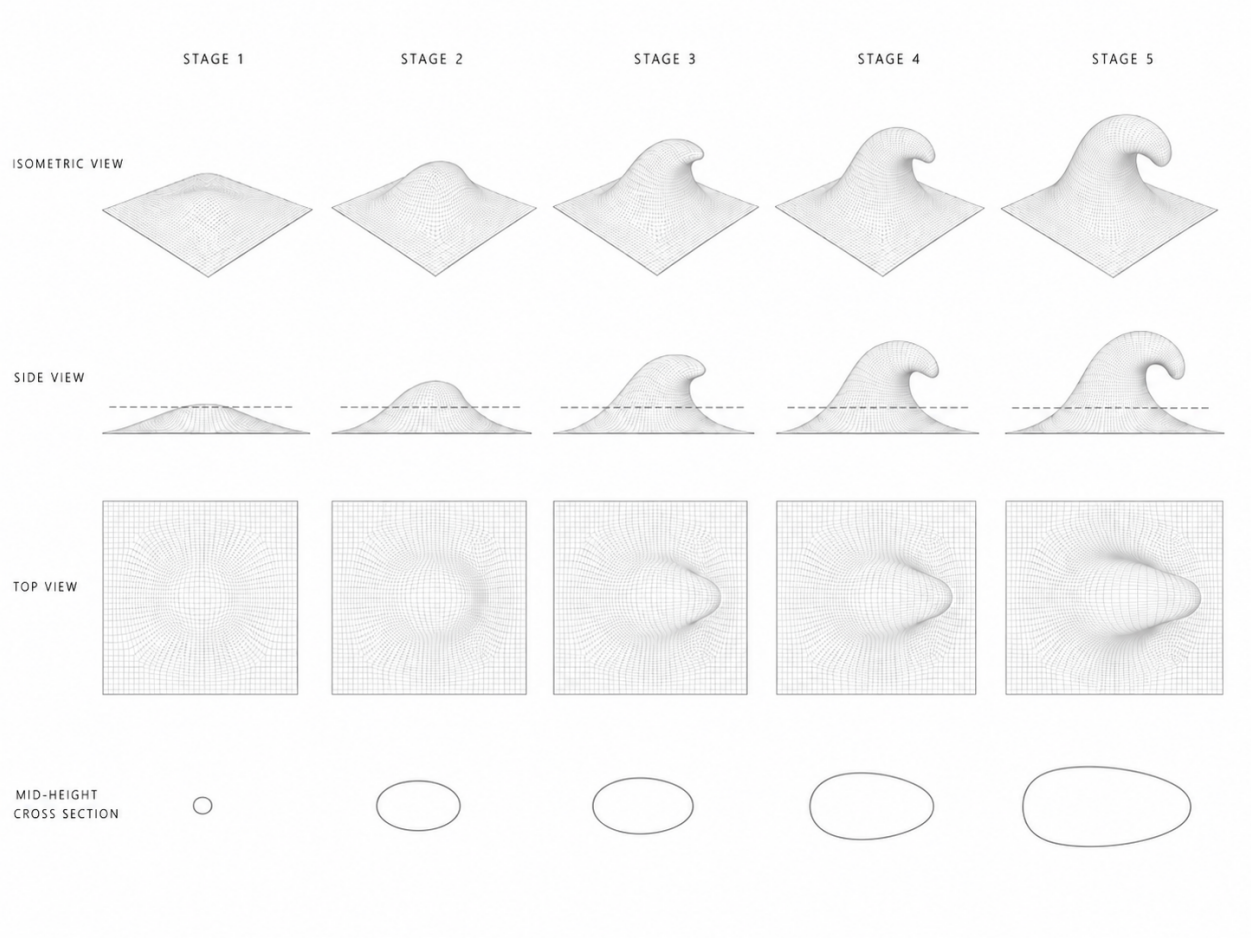

Figure 5. June 28 supplied stage sequence with an explicit mid-height cross-section row. The bottom row tightens the plan-footprint target: the active body should grow into one smooth oval or teardrop-like section through the curl, not a pointed spear, pinched hourglass, terminal stripe, or pasted helper contour. This reference is now part of the WaveVis acceptance set.

2 Start here for a new WaveVis chat

If this PDF is the only WaveVis context available, start with this section and then inspect the live artifact. The current project state is:

- **Verified public route:** `https://aolabs.io/wavevis/`. This route serves the fallback bundle that Alan actually uses.
- **Preferred but blocked route:** `https://wavevis.aolabs.io/`. At the time of this record, HTTPS validation can fail because the custom-domain certificate is not valid for that hostname. Do not treat the custom subdomain as the verified route until a fresh certificate check passes.
- **Visible PDF action:** the WaveVis header links to the paper file under `/proofs/`. File:

`wavevis-system-architecture.pdf`. The old connector proof remains a source artifact, not the primary user-facing paper action.

- **Current source root:** workspace path `_apps/wavevis.aolabs.io/`.
- **Public fallback mirror:** workspace path `_apps/aolabs-site/wavevis/`. A WaveVis deploy to its own `gh-pages` branch does not automatically update the `aolabs.io/wavevis/` fallback unless the built files are mirrored there and the hub site is pushed.
- **Current architectural direction:** curated custom Moana-ocean profile localized into the inverse-sheet field. Manual xz and xy profile editors are not the active path.
- **Current reference-match boundary:** the June 30, 2026 Candidate710 checkpoint targets the June 23 line drawing, the June 24 smooth gridded breaking-wave sequence, and the June 28 stage/cross-section reference. The default product view shows the readable surface plus the connected X-cell mechanism: surface is on, `X legs` are on, and `cells` are on. Reference-mode URL gates remain addressable for inspection, but they cannot suppress the mechanism layer: a request such as `surface=1&connectors=0&cells=0` is forced back to the full mechanism view so no public capture can become a surface-only silhouette. Side is a full three-dimensional side camera with visible cells, legs, shared rods, and spherical connector joints, not a profile-only debug view and not a surface-only silhouette. Candidate710 keeps the no-cavity tapered-body acceptance, terminal taper acceptance, perimeter-strip acceptance, and full-mechanism visibility acceptance: the literal profile, sampled rendered edge, sampled perimeter border band, sheet footprint, and visible linkage layer must all survive before a screenshot can be treated as useful evidence. Candidate710 preserves the view-aware readable frame: Side and 3D/isometric use a smoothed reference-leaning curled profile while Top and Front keep the safer rounded proof profile so the plan proof does not inherit the side curl opening. The target grid still runs `blockSurfaceBowlPockets` before morphing and node emission; any forbidden local low point is lifted out of the model before browser screenshots, figure export, or deployment can use it. The current hard mechanism gate is exact cell-level X geometry: every interior cell is one black square body with four straight legs, opposite leg pairs are equal length and collinear through the cell center, the cell center bisects both bars, the two pair lengths and included angle may differ, and every spherical connector node is a physical two-cell joint with exactly two attached leg uses. The current X mechanism reports 4928 black square cells, 19400 rendered center-to-connector legs, 19400 shared-joint arms, 9700

spherical physical connector nodes, 9544 checked opposite pairs, interior leg-count range 4–4, zero connector endpoint gap, zero opposite-pair length spread, zero opposite-pair collinearity error, zero opposite-center residual, physical connector use count 2–2, and 0 over-occupied physical connectors. Connector corridor drift is now a bounded diagnostic, not the pass condition. The default dense wave remains 44×112, while URLs may reduce columns to 12 or more for X-cell inspection. The checkpoint is guarded mechanism/readability progress, not a completed reference match: the perimeter-lift and cavity/dimple families are blocked in source checks, the 79-state slider sweep reports no bad cases, the terminal taper gate passes, the top view no longer collapses into a triangular fan, and all product cameras keep the full mechanism visible, but the side curl remains sparse and approximate against the supplied tube/cross-section reference.

2.1 What the next agent must not repeat

The repeated failures that this document is meant to prevent are:

1. Do not make a swept tunnel, ribbon, canopy, roof, half-pipe, bridge, or extruded strip.
2. Do not form a bowl, dimple, cavity, pucker, or side wall that covers the underside of the curl.
3. Do not replace Side with a profile-only debug view. Side is a side camera view of the full three-dimensional sheet.
4. Do not hide, ghost, clip, or remove linkage cells because they look bad. Fix the geometry.
5. Do not remove the full array, one-center X cells, exact collinear equal opposite pairs, two-use physical connector joints, or two-cell linkage truth while chasing a prettier outline.
6. Do not wire the lip directly to the ground or draw filler geometry across the open underside.
7. Do not make `position` or `steer` shape controls. They are rigid post-shape transforms.
8. Do not call the work done from code intent, automated checks, a nicer profile line, or a local-only screenshot.
9. Do not let the top view collapse into a big triangle, wedge, or V fan while the side view looks acceptable.

10. Do not paste a mid-height contour, helper ellipse, bridge, or cross-section overlay into the product view to fake the June 28 cross-section row. Fix the surface projection and preserve the X-only proof path.

2.2 Current live-state evidence

Figures 6–13 record Candidate710 from the current browser-rendered source tree. The default presentation is a readable gridded surface plus the connected X-cell mechanism inspired by the June 24 and June 28 references. Candidate710 separates the readable display profile by view: Side and 3D/isometric receive the smoothed reference-leaning curled profile, while Top and Front keep the safer rounded proof profile so a side-shape repair does not turn the plan view into a triangular fan. The checker blocks the specific failures Alan named: perimeter lift, bowl, dimple, cavity, lower-branch rebound, lower-branch backtrack, hooked terminal descent, cliff-like terminal drop, surface-local bowl pockets, top-view triangular fan collapse, and silhouette-only mechanism visibility. The source blocker runs before rendered evidence: `blockSurfaceBowlPockets` lifts forbidden local minima in the target grid; the readable perimeter sampler rejects lifted edge lines, lifted perimeter border bands, footprint escape, terminal pinch, and non-straight terminal boundaries; the terminal taper sampler rejects hook/cliff finishes; and slider robustness checks 79 cases with no side-cavity or surface-bowl bad cases. Front remains a separate spanwise check with the same black-square cell and spherical-node rendering. Top keeps one square sheet panel, one continuous X-frame layer, a stronger reference-sheet grid, a top-only height-shadow derived from the readable surface, and a rounded footprint instead of inheriting the side-only curl opening. The mechanism is not deleted: `X legs` exposes the actual collinear equal-pair X cells, `surface=0&connectors=1&cells=1` exposes the X-only mechanism view, `surface=1&connectors=0&cells=0` is now a mechanism-locked surface request rather than a valid surface-only acceptance view, and `node scripts/check-inverse-sheet.cjs` verifies 4928 black square cells, 19400 rendered center-to-connector X legs, 19400 rendered shared-joint arms, 9700 spherical connector nodes, 9544 checked opposite pairs, interior leg-count range 4–4, zero connector endpoint gap, zero opposite-pair length spread, zero opposite-pair collinearity error, zero opposite-center residual, exactly two physical endpoint uses per connector, 0 over-occupied physical connectors, `maxEdgeZ=0`, `maxBorderBandZ=0`, `maxPlanXOvershoot=0`, `sourceUsesBlackSquareCells=true`, and `sourceUsesCollinearEqualPairConnectors=true`. Candidate710 keeps a 12×12 proof route so the same four-leg, two-use connector contract is readable without the dense whole-array overplot. This paper therefore treats the current render as verified mechanism and perimeter progress with an

open visual-match boundary: all public cameras show the mechanism, but the full reference match remains sparse and approximate.

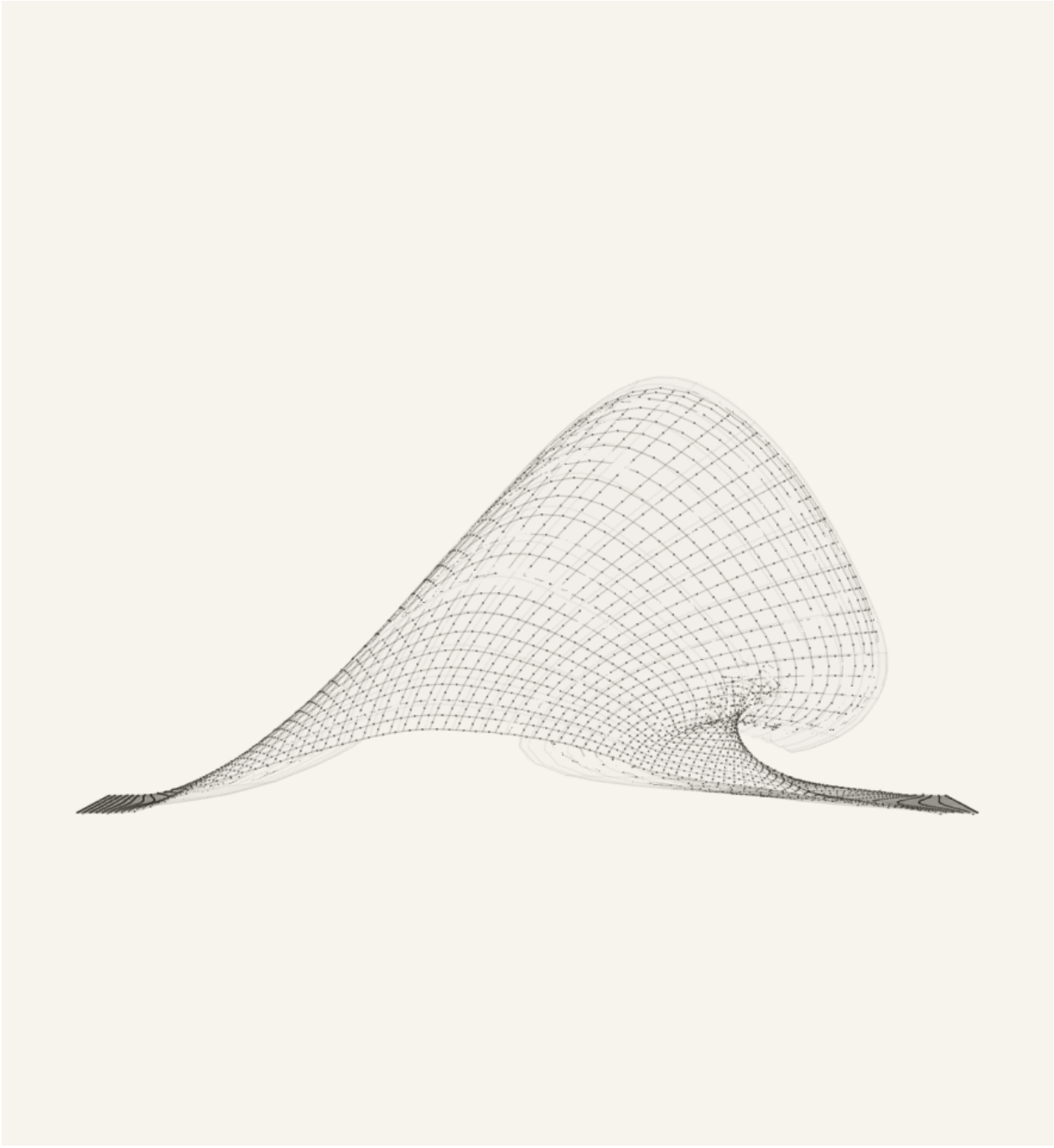

Figure 6. Candidate710 browser-rendered Side view for the default inverse-sheet state. The view keeps the readable surface subordinate to the actual mechanism: a lighter surface skin, pale side wire grid, black square one-center/four-leg cells, spherical two-use connector nodes, and stronger instanced shared-joint rods are visible together. The underside no longer contains the rejected bowl/dimple/cavity family, the perimeter border band is source-gated to stay flat, and Side uses a view-specific smoothed curled profile rather than the capped Candidate661 side path. The reference-wave match is still open: this checkpoint closes the surface-silhouette failure, but it remains sparse and approximate rather than a completed Moana-like plunge.

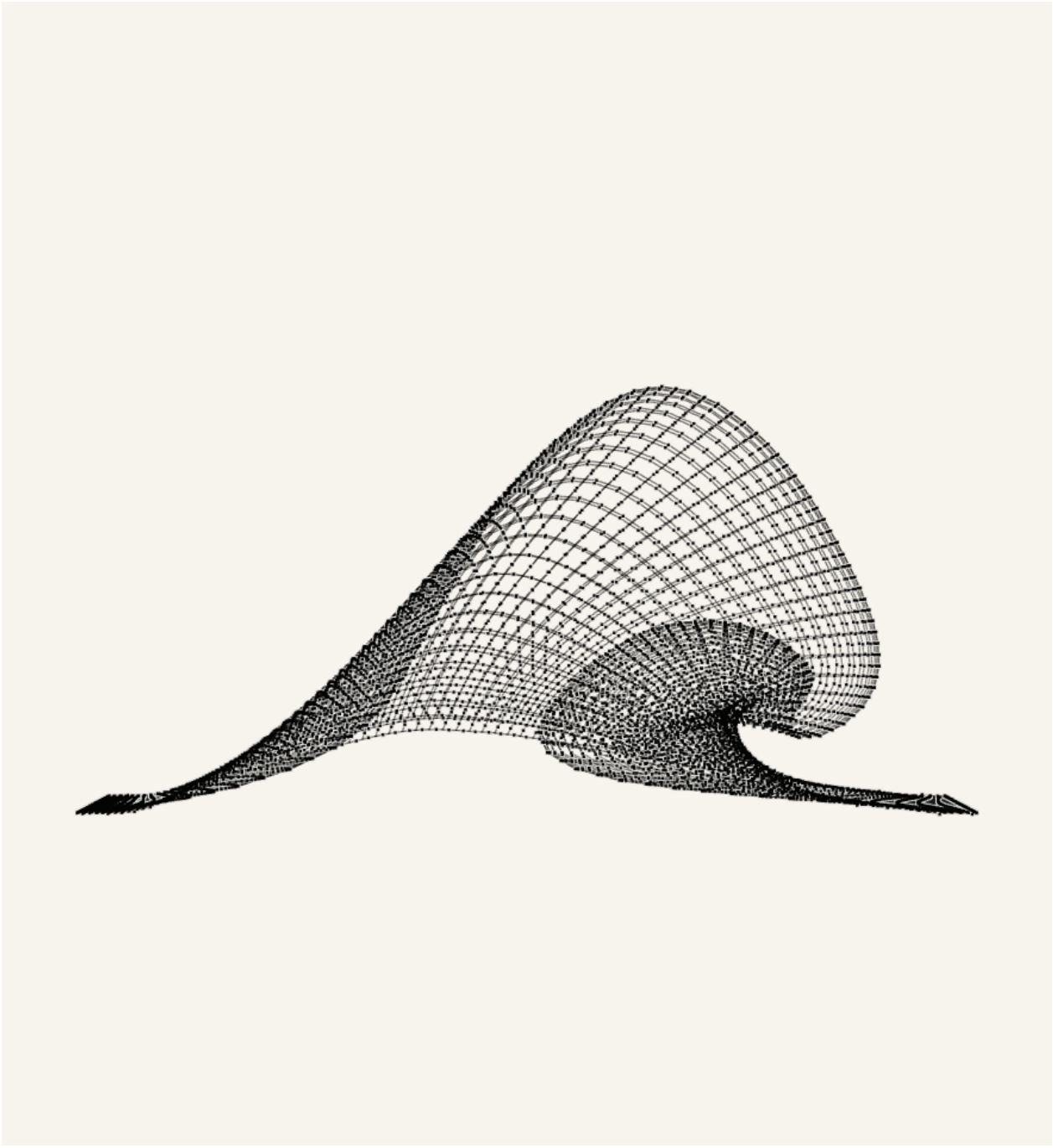

Figure 7. Candidate710 Side view with `surface=0&connectors=1&cells=1`. This X-only evidence removes the surface skin and shows the actual mechanism layer after the side readable-profile split: each cell is one black square with four straight center-to-connector legs, opposite pairs are equal length and collinear through the cell center, and every connector is a spherical physical node with exactly two attached leg uses. Neighboring cells meet through visible connector nodes; no between-cell connector is allowed to become an X or a four-cell joint.

Figure 8. Candidate710 browser-rendered 3D/isometric view for the same default state. The default presentation retains the recovered geometry floor, a lower side-biased isometric camera, smoothed curled readable surface skin, pale wire grid, black square four-leg X cells, spherical two-use connector nodes, and stronger mechanism ink than the rejected silhouette pass. The 3D view is still a mechanism view, not a surface-only substitute. This figure remains an improved open visual-match boundary rather than an exact reference match.

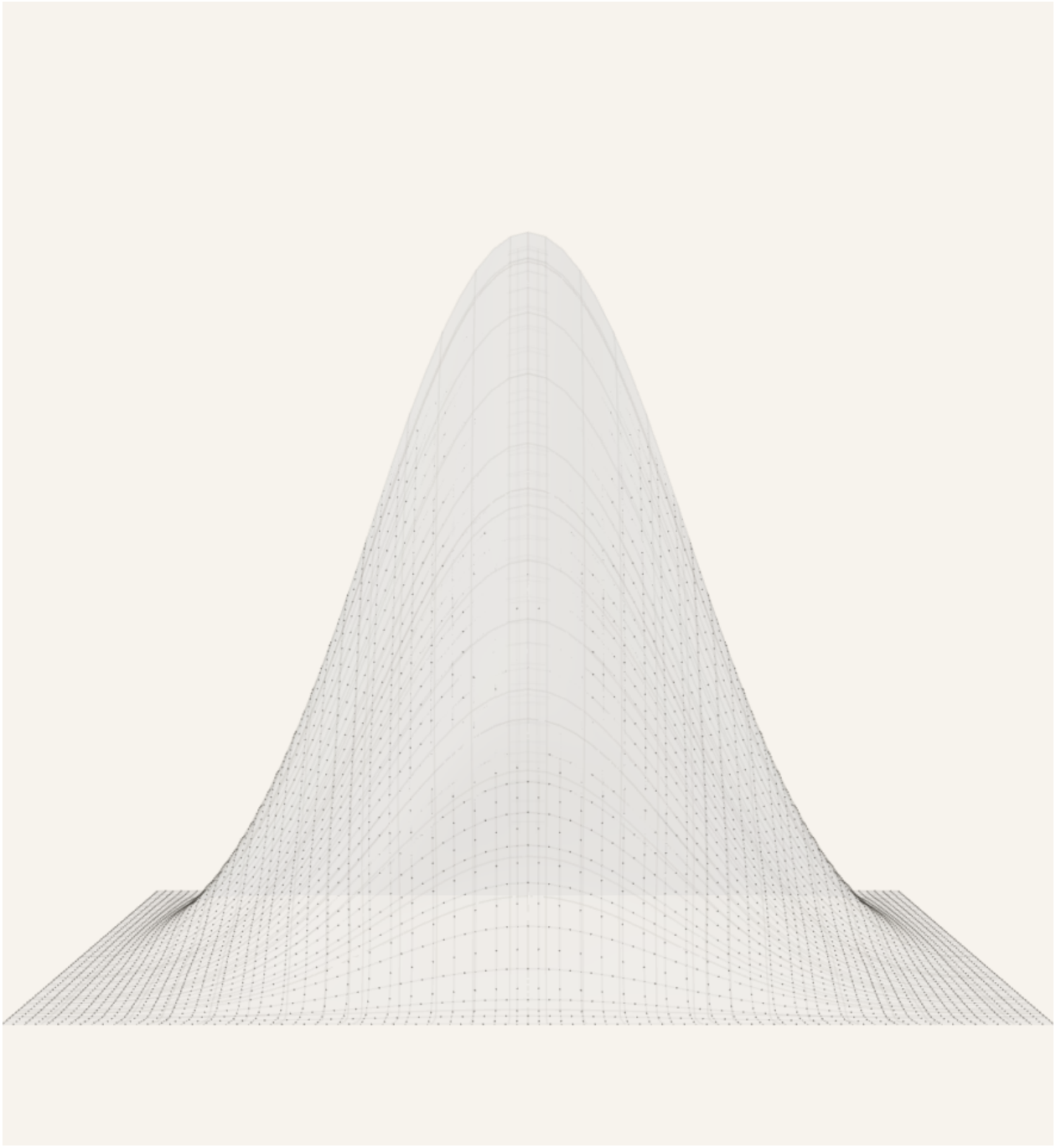

Figure 9. Candidate710 browser-rendered Front view retained as the separate spanwise product camera. This view compresses front depth, bounds the terminal lip contribution, depth-tests the pale grid against the lip, keeps rounded terminal width, and shows the black-square X-cell linkage with spherical connector nodes through the projected cap rather than hiding mechanism state behind a pasted cap layer. The projection checkpoint remains open against the June 24 and June 28 references.

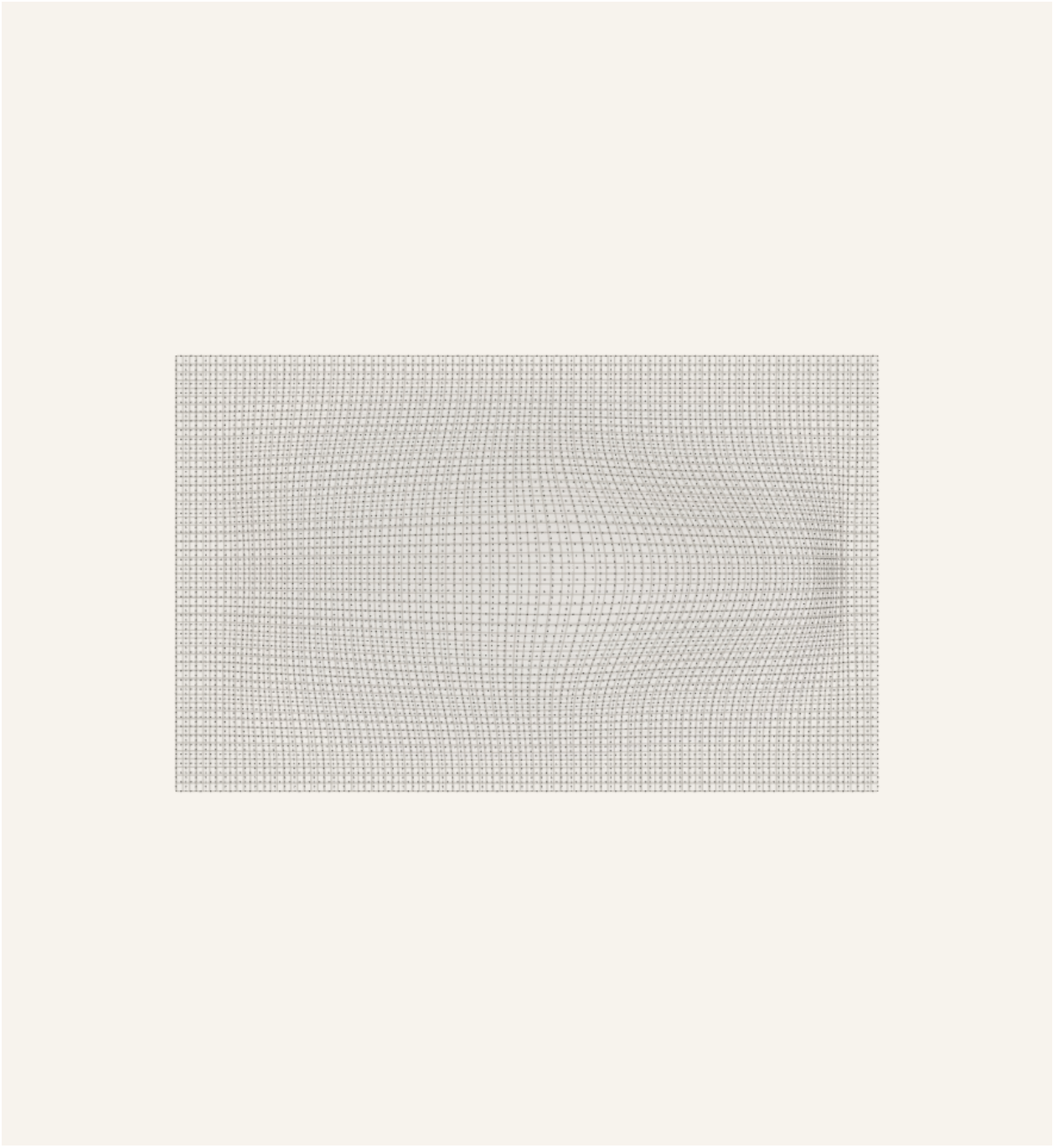

Figure 10. Candidate710 browser-rendered Top view for the same default state. The surface-plus-X view keeps the rounded proof profile rather than inheriting the side-only curled profile, and it uses a stronger reference-sheet grid, one continuous black-square/four-leg X layer, real center-to-connector rods, spherical connector nodes, a guarded mid-section oval term, and a bounded height-shadow from the readable surface. Strict `surface=0&connectors=1&cells=1` top inspection still exposes the collinear equal-pair X-only mechanism rather than hiding it, and the top footprint remains an open visual-match target rather than a solved reference state.

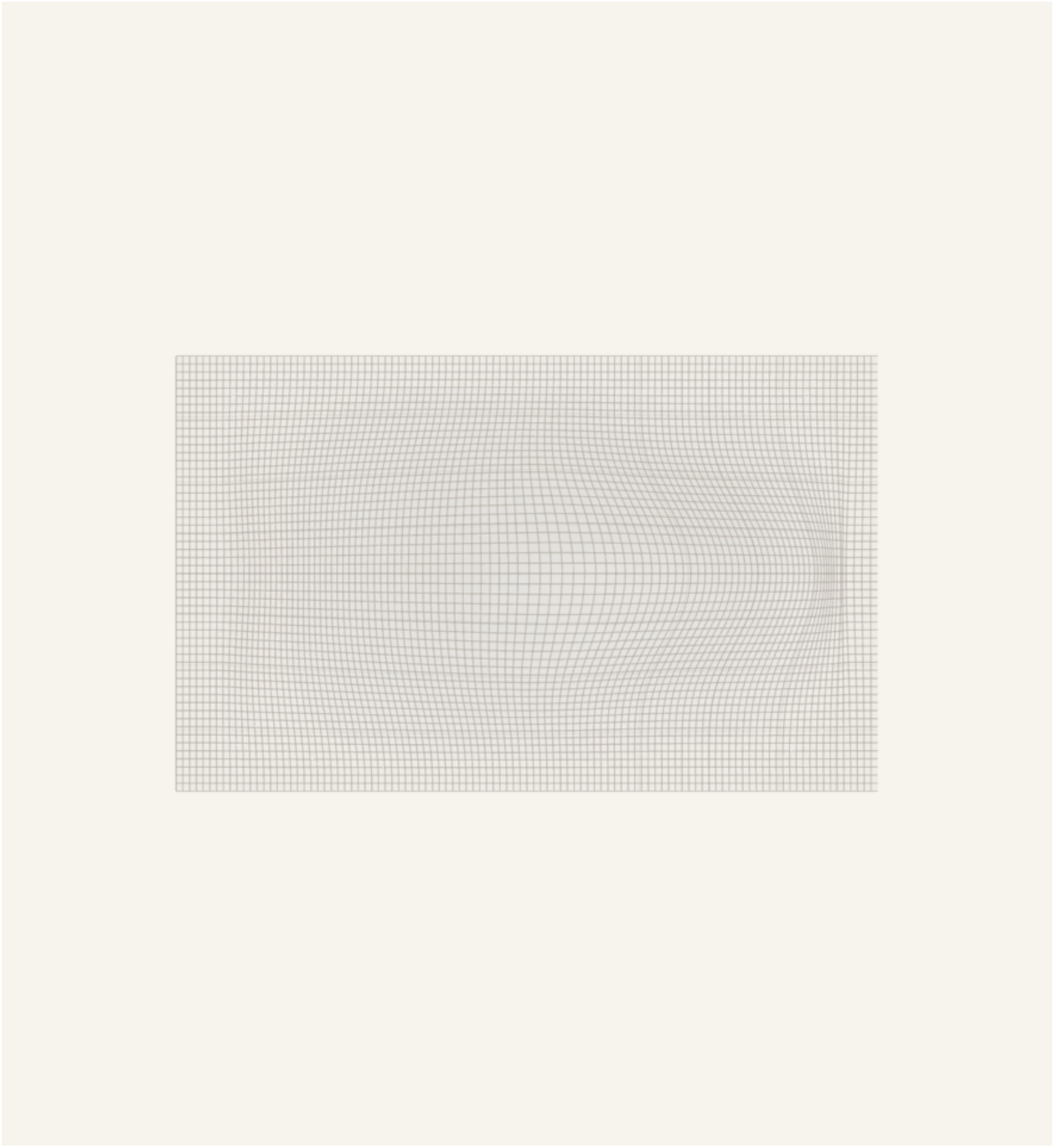

Figure 11. Candidate710 Top view after a `surface=1&connectors=0&cells=0` request. Reference mode no longer allows a surface-only acceptance state: the readable surface request is still mechanism-locked, so the full X-cell sheet stays visible while the bounded height-shadow remains derived from the Top readable wave surface.

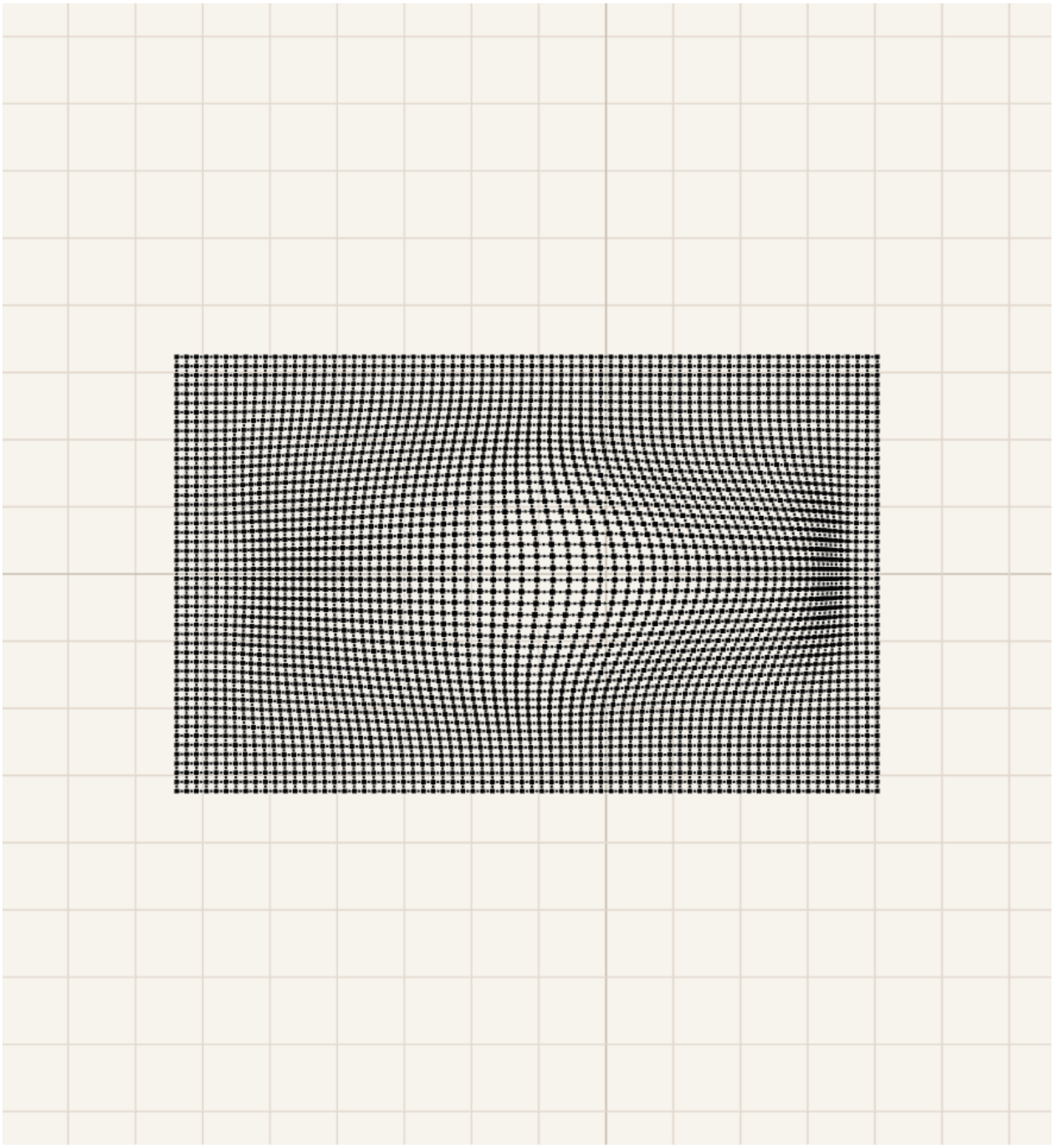

Figure 12. Candidate710 Top view with `surface=0&connectors=1&cells=1`. This proof view is intentionally dense because it shows the whole array through the `X legs` and `cells` layers. Each visible black square cell is the body for four straight legs, each opposite pair remains collinear/equal through the center, and adjacent cells meet at spherical two-use connector nodes. This is the current hard mechanism acceptance test, not a decorative overlay.

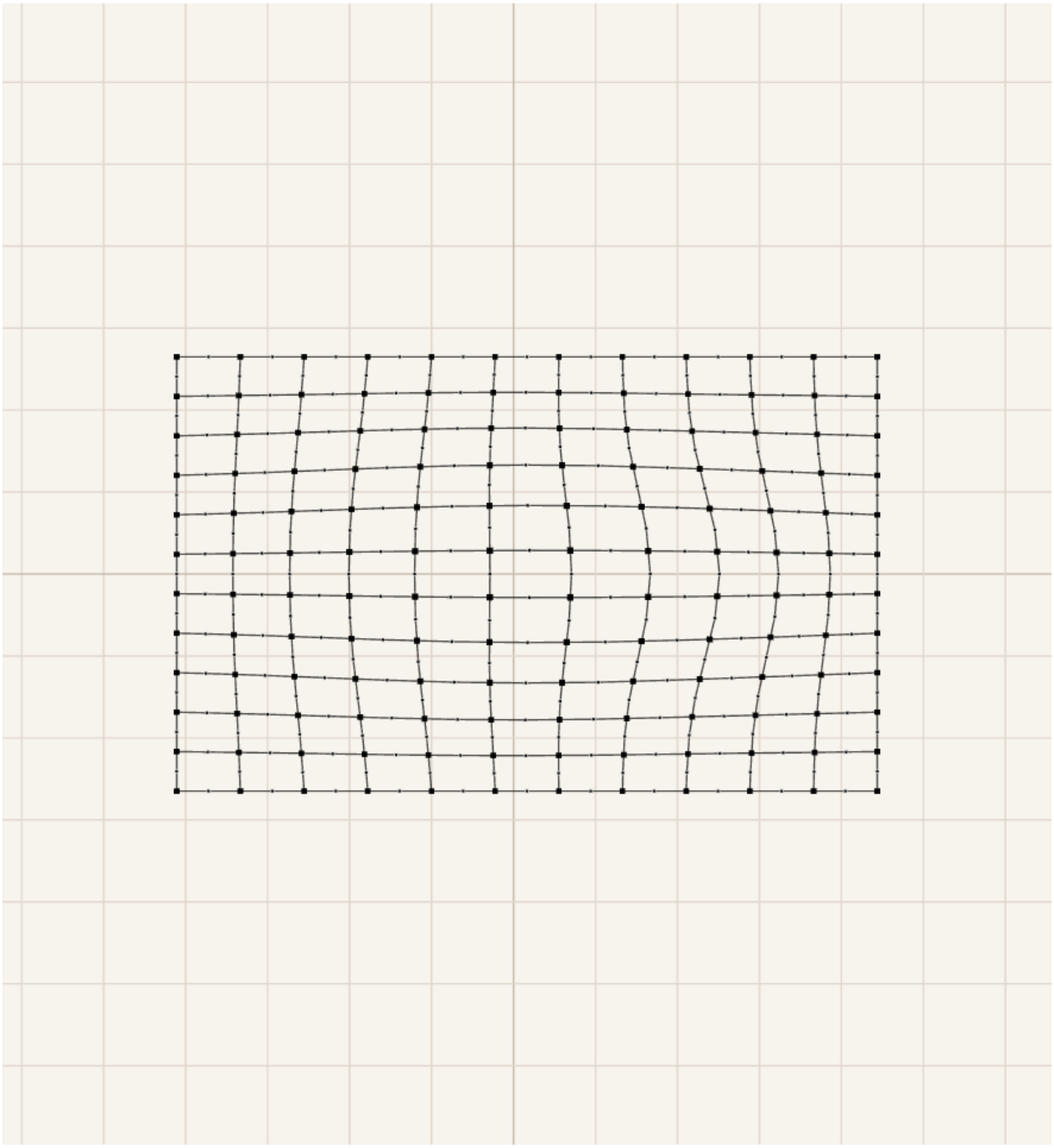

Figure 13. Candidate710 inspectable Top proof with `rows=12`, `columns=12`, `surface=0`, `connectors=1`, and `cells=1`. This low-density route exists so the hard cell contract can be inspected directly through the `X legs` and `cells` layers: a cell is not two nearby two-leg elements; it is one black square with four straight legs, opposite pairs are collinear and equal through the center, and each visible connector is a spherical node used by exactly two neighboring cells.

2.3 File map

File or route	Ownership
`src/latticeGeometry.ts`	Source of truth for the inverse sheet model, defaults, sanitization, target displacement field, morph behavior, metrics, and geometry sanity checks.
`src/LatticeViewer3D.tsx`	Source of truth for Canvas rendering, side/isometric/top full-sheet camera behavior, surface visibility, camera presets, visual layers, and the rendered collinear equal-pair X-cell and shared-joint layers.
`src/xCellMechanism.ts`	Source of truth for the connected X-cell display mechanism: one black-square cell body per lattice cell, four straight center-to-connector legs per cell, exact collinear/equal opposite pairs through the cell center, and spherical connector nodes with exactly two attached leg uses.
`src/TargetShapeControls.tsx`	Source of truth for the visible inverse-sheet controls: rows, columns, views, scale, morph, position, steer, display toggles, reset, export, and import.
`src/InverseSheetTab.tsx`	Source of truth for URL-state loading, tab order, view requests, import/export plumbing, and the Inverse Sheet first-tab behavior.
`scripts/check-inverse-sheet.cjs`	Main inverse-sheet regression checker. It compiles the TypeScript geometry and checks parameter semantics, lip curl, flatContribution, low-lip stability, lateral smoothness, no bowl pockets, terminal taper, startup URL coverage, and the collinear equal-pair X-cell render contract.
`scripts/check-geometry-invariants.cjs`	Mechanism-level invariant checker for rigid-cell/linkage behavior.
`proofs/wavevis-system-architecture.tex`	Source for this PDF. Update it when the public simulator contract, current state, or known failure modes change.
`proofs/figures/`	Moana reference images, mechanism proof figures, and current source-rendered state figures used by this paper.
`dist/`	Built WaveVis static app after `npm run build`. Do not edit by hand.
`aolabs-site/wavevis/`	Public fallback mirror for `https://aolabs.io/wavevis/`. Copy the built `dist` contents here when the public fallback must update.

Table 1. Current source map. A new WaveVis chat should use these files rather than reconstructing behavior from old prompts.

2.4 Minimal continuation workflow

A fresh agent continuing WaveVis should do the following in order:

1. Read this PDF and `src/latticeGeometry.ts`, then inspect `src/LatticeViewer3D.tsx` and `src/TargetShapeControls.tsx`.
2. Open the verified public route `https://aolabs.io/wavevis/` and the current local route if a dev server is running.
3. Reproduce Alan's active preset from the URL before changing source. Do not judge a different default state.
4. Run `npm run check:geometry`. If it fails, fix source before visual tuning.
5. Render and inspect `Side`, `3D`, and `Top`. Side and 3D must both be full-sheet views with the curl visible.
6. Make the smallest source change that improves the human-facing Moana-wave outcome without regressing the mechanism invariants.
7. Re-run `npm run check:geometry` and `npm run build`.
8. Verify live/public output after deploy or fallback mirror update. A local screenshot is not the public artifact.
9. Update this PDF if the architecture, current state, source map, controls, or known failure classes changed.
10. Post a Progress event with complaint, issue, Codex change, observed source change, provenance, and commit ids.

3 Current implementation state

3.1 Defaults and target

The current inverse-sheet default is intentionally taller and sharper than earlier flat presets:

$$\begin{aligned}
\texttt{rows} &= 44, & \texttt{columns} &= 112, & \texttt{morph} &= 1,\\
\texttt{height} &= 15.25, & \texttt{horizontalOffset} &= 22.5, & \texttt{overhangWidth} &= 32,\\
\texttt{overhangAngleDeg} &= 120, & \texttt{overhangPosition} &= -0.15, & \texttt{profileScale} &= 0.55,\\
\texttt{smoothing} &= 1, & \texttt{lipSharpness} &= 0.88, & \texttt{wallSmoothness} &= 0.28,\\
\texttt{flatContribution} &= 0.35.
\end{aligned}$$

The app displays only the narrow active controls by default. Legacy inputs such as `height`, `overhang`, `lipDip`, `width`, `lipSharpness`, `groundTransition`, `wallSmoothness`, and `flatContribution` can still be parsed from URLs and JSON imports because older test links and screenshots use them. The visible startup path favors `scale`, `morph`, `position`, and `steer` to keep the main wave profile stable.

The default reference-wave sheet remains `columns` = 112. The sanitizer now allows `columns` ≥ 12 so X-cell proof routes can be inspected at low density, but compact column counts are not evidence of an exact reference-wave match. Dense-wave regression checks request the default column count explicitly; low-column screenshots are mechanism-readability proofs.

3.2 Target-field construction

The target is constructed in a canonical wave frame. The canonical frame removes `position` and `steer`; the shape is computed there; then steer and position are applied as rigid transforms afterward. This prevents the old bug where moving the wave also changed its profile.

The source sequence in `buildNodes` is:

1. build rest grid p^0_{ij};
2. compute core target positions from `targetFromDeformationField`;
3. pin perimeter nodes to rest;
4. apply span continuity smoothing for generated mode;
5. apply flat-contribution diffusion as a support apron, not as a flattening blend;

6. interpolate from rest to target with `morphGrowthAmount`;

7. build metrics, edges, quads, and dihedral pairs from the resulting grid.

The target function samples u along the wave field and v across span:

$$u = \frac{x - x_{\min}}{L}, \qquad v = \frac{y + S/2}{S}.$$

The displacement is zero outside the support field and on the perimeter. The support field expands with `smoothing` and `flatContribution`, but the core wave must remain the same shape.

3.3 Rigid controls

`position` and `steer` are post-shape placement controls:

$$\Delta x_{\text{position}} = 0.045\, L\, \text{clamp}(\texttt{position}, -1, 1),$$

$$\psi_{\text{steer}} = 45^\circ\, \text{clamp}(\texttt{steer}, -1, 1).$$

They must not modify the curated wave profile, width, curl, strain, or topology. Validation aligns geometries back to the canonical frame and checks that residual shape differences are near zero for position and bounded for steer.

3.4 Morph control

`morph` is not a shape recipe. It interpolates from rest to the completed target:

$$p_{ij}(m) = (1 - g(m))p^0_{ij} + g(m)p^*_{ij}.$$

The current growth function combines a sine ease with a faster visible growth curve so the wave grows naturally without a dead early range. Morph must preserve topology, connector closure, and the visible wave identity throughout the range. If `morph` = 0.5 produces zig-zags, overlaps, missing cells, or a side-wall-covered curl, that is a geometry failure.

3.5 Flat contribution and ground transition

`flatContribution` does not mean flatten the wave. It means share deformation with the surrounding sheet through a support apron. In accepted behavior:

- the main wave height and overhang reach stay within a small residual;
- the centerline profile remains essentially unchanged;
- surrounding flat areas participate more gradually;
- strain concentration is reduced or spread rather than hidden.

The old behavior `target` = lerp(`deformed`, `rest`, `flatContribution`) is explicitly rejected because it turns `flatContribution` into a flattening slider.

`smoothing`, shown historically as ground transition, broadens the support field and root ramp. It should recruit surrounding flat sheet into the slope while keeping the perimeter pinned and preserving the terminal lip.

3.6 Lip dip and terminal curl

`lipDip` maps to `overhangAngleDeg`. Neutral is 90°. High curl is 120°. The accepted behavior is not endpoint lowering. The wave should rise, crest, reach forward, curl downward, then return smoothly to the flat sheet without closing into a bowl. The current validation checks:

- the selected lip nose is a mid-height visible point below the last peak, not the near-floor return;
- the terminal slope is negative;
- the tip is forward of the crest and tucked under the shoulder by a bounded amount;
- the return advances toward the flat front instead of folding backward;
- `terminalSideProfileCavityBlock.ok`, `noBackfoldCavity`, and `noVisibleDimplePocket` remain true.

These checks are guards, not substitutes for the human-facing screenshot. A visible dimple, pocket, side wall, or missing curl still means open work.

3.7 Rendering contract

`LatticeViewer3D.tsx` renders the readable surface and a connected X-cell mechanism from the actual lattice graph. In the current contract:

- `view=side` sets a side camera over the full three-dimensional linkage array;
- `view=front` sets a front camera over the same full three-dimensional sheet so the spanwise lip cap can be inspected;
- `view=isometric` shows the full rectangular sheet in perspective while still making the curl and lip readable;
- the default product view uses a smooth gridded surface skin plus one-center four-leg X-cell leg lines, black square cell bodies, and spherical connector nodes;
- Side, Front, and 3D use view-specific mesh outlines instead of the previous SVG overlay or red terminal trace;
- each rendered X cell is one black square cell body with four visible center-to-connector legs;
- each connector is a spherical two-use physical node between adjacent cells;
- each connector has exactly two endpoint uses: one leg from each of those two centers;
- crossing diagonal families must stay visually and physically separate instead of collapsing into a four-cell joint;
- connector corridor drift is reported as a bounded diagnostic when exact cell-level opposite-pair closure moves a connector away from the old midpoint corridor;
- cells and X legs are display toggles, not repair paths, and their render scope remains covered by the geometry check.

If the renderer needs to hide rows to make the shape readable, the generator is still wrong.

(a) Open underside and suspended lip.

(b) Forward/downward curl at the water edge.

(c) Localized mound emerging from flat water.

(d) Rounded column without a hard perimeter wall.

Figure 14. Additional supplied reference frames. Together they define the acceptance target: water-like surface continuity, strong center curl, smooth lateral fade, no side wall covering the underside, and no swept tunnel extrusion.

4 Target outcome

4.1 Human-facing shape

The accepted target is a “flat sheet plus localized plunging wave”:

- the outer perimeter of the rectangular sheet remains flat and fixed;
- the wave emerges through a broad, smooth ramp rather than a vertical wall;
- the crest is rounded, with no top dent;
- the lip projects forward and curls downward;
- the open underside is visible from side view, not covered by a side wall;
- the shape fades back to flat in every direction;
- the array remains a full linkage field with nodes and legs, not a hidden or clipped shell.

Figure 15 gives the practical acceptance boundary. The shape must read as a local wave in a sheet, not as an extrusion. A single side profile can guide the curl, but the generated three-dimensional field must vary across span.

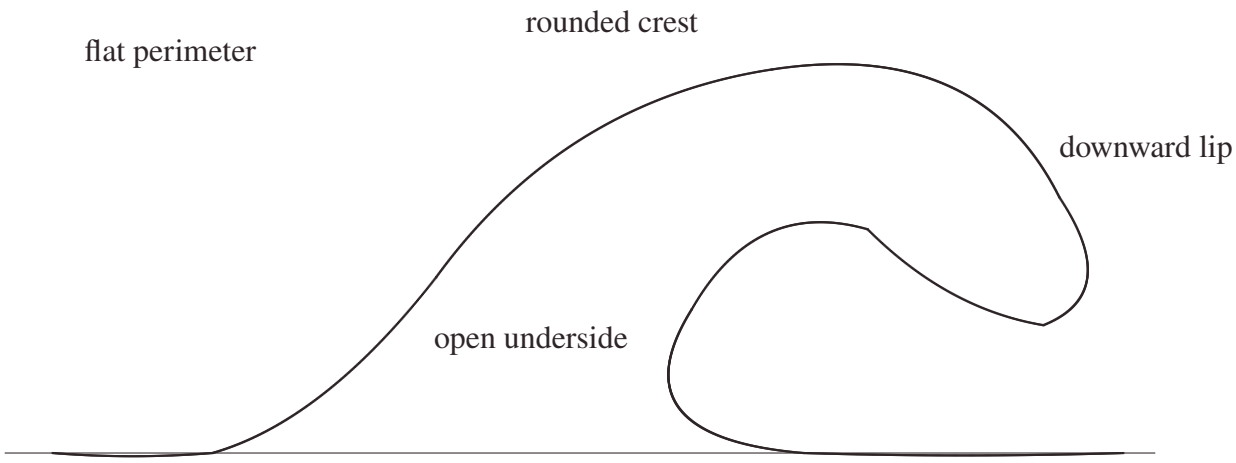

Figure 15. Desired side-profile shape as one constraint inside the full mechanism view. The drawing is not a literal mesh recipe and not an accepted output by itself. It defines a smooth back ramp, larger outer radius, smaller inner radius, curled lip, and curved interior throat; the three-dimensional solver must localize that profile in the middle of a full visible linkage sheet rather than sweep it as a constant tunnel or collapse the render to a trace.

4.2 Hard constraints

The WaveVis generator and renderer must preserve the following hard constraints:

1. **Flat perimeter.** Boundary nodes must remain on the rest sheet. No side, front, or rear perimeter should be pulled into the wave.
2. **Full array visibility.** The rendered artifact must show the actual linkage grid. Hiding or ghosting broken cells does not count as a fix.
3. **Connector closure.** A cell connector is an actual shared graph location. Lines should meet at nodes; apparent detached legs are a failure.
4. **No erratic legs.** Long arms, spikes, zig-zags, random backtracking, and visible overlaps indicate a broken geometry or render path.
5. **Smooth strain sharing.** Adjacent rows and columns should change gradually enough that the mesh reads as a continuous mechanism field.

6. **Localized wave.** The curl must be strongest near the center and fade laterally. A constant side-to-side tunnel is not the target.

7. **Full-view side semantics.** Side view shows the full three-dimensional render from the side. It must not be replaced by a profile-only debug view.

8. **No filler geometry.** Cosmetic walls, caps, planks, bridges, hidden couplers, or masks must not be used to conceal a bad mechanism.

5 Coordinate model

The simulator starts from a rectangular rest lattice. A node with row i and column j has rest position

$$p_{ij}^0 = (x_j, y_i, 0).$$

The target is a displacement field

$$p_{ij}^* = p_{ij}^0 + m\, D(x_j, y_i; \theta),$$

where $m \in [0, 1]$ is the morph amount and θ represents all shape parameters except rigid placement. The morph parameter only interpolates between rest and the finalized target. It must not change the target shape definition, remesh the lattice, or introduce a different topology.

The longitudinal coordinate $u \in [0, 1]$ measures progress through the wave region from rear/root to front/lip. The span coordinate $v \in [-1, 1]$ measures lateral distance from the centerline. A good WaveVis target uses both:

$$D(x, y) = A(u, v)\, P(u) + B(u, v)\, Q(u),$$

where $P(u)$ is the side-profile contribution, $Q(u)$ can include forward curl or secondary displacement terms, and A, B are smooth two-dimensional envelopes that go to zero at the sheet perimeter.

The key architectural rule is that $A(u, v)$ is not constant in v. A constant span envelope turns any attractive profile into a tunnel. A localized plunging wave requires $A(u, 0)$ large near the centerline and $A(u, \pm 1) \approx 0$ near the sides, with a monotone or smoothly unimodal falloff rather than a hard wall.

6 Generation pipeline

The target-building pipeline is:

1. sanitize rows, columns, visible sliders, hidden legacy shape parameters, display flags, and URL parameters;
2. build the full rectangular node grid;
3. compute the canonical Moana-ocean wave displacement from the curated target;
4. apply the span envelope that localizes the wave in y;
5. preserve the boundary by forcing perimeter displacement to zero;
6. apply morph interpolation from rest to target;
7. apply rigid steer and position transforms after the shape is finalized;
8. build full rectangular edge and quad topology;
9. compute strain, bend, shear, dihedral, and display metrics;
10. render the full array, side view, 3D view, and top view without substituting fake surfaces.

The distinction between the curated target and rigid placement parameters is essential. The visible app no longer exposes manual xz or xy profile editors. The source target owns the Moana-ocean profile. The user-facing sliders are intentionally narrow: scale uniformly resizes that target, morph blends from rest to the completed target, steer rotates the finished target around the anchor, and position translates the finished target horizontally. Neither steer nor position may feed back into profile curvature, curl, width, strain generation, or cell topology.

Stage	Contract	Failure caught
Target profile	Defines the desired side profile shape.	Blob, neck/head, hill without lip, top dent.
Span envelope	Localizes the wave in the sheet.	Swept barrel, side wall, U-shaped tube blocker.
Boundary clamp	Keeps the perimeter flat.	Pulled sheet edges, non-flat perimeter.
Morph	Interpolates rest to target only.	Half-morph mangling, topology change while sliding.
Topology	Keeps full node/edge/quad graph.	Missing grid, hidden cells, detached legs.
Renderer	Shows the actual mechanism.	Fake shell, hidden wall, side view as profile-only debug output.

Table 2. Pipeline stages and the failure modes each stage must prevent.

7 Curated Moana Target

The current visible app does not ask the user to draw a profile. That path created too much ambiguity and repeatedly made the simulator look like a swept tunnel or hand-edited cavity rather than a single coherent wave. The user-facing target is now driven by a curated custom Moana-ocean lip/body profile: a localized ocean deformation with a broad rear ramp, rounded crest, forward-and-down lip, open underside, smooth lateral fade, and flat rectangular perimeter.

The curated target is a reference field, not a set of cell locations. The final cell grid is determined by row and column counts, then sampled against that target. A clean state must keep the full linkage array visible while making the centerline and lateral profile read as one plunging wave. The target must never stitch the lip directly to the ground or draw a hard polygon boundary around a former editor curve, because that creates the wall, bowl, cavity, and silhouette-only failures this record explicitly rejects. The curl is also resolution-sensitive: the default reference-wave state keeps 112 columns, while reduced column counts are reserved for low-density mechanism inspection and must not be mistaken for a completed reference-shape view.

8 Slider semantics

Control	Intended effect	Must not do
`morph`	Blend rest sheet into the completed target.	Remesh, flip cells, create zig-zags, change profile identity.
`scale`	Uniformly enlarge or reduce the curated Moana-ocean target.	Change steer, position, topology, or the target's wave identity.
`steer`	Rigid yaw rotation of the finished wave.	Bend, stretch, or reshape the wave.
`position`	Rigid horizontal translation of the finished wave.	Change profile, strain field, or curl.
display toggles	Surface, flat grid, cells, and X legs are display layers only.	Hide broken geometry or substitute a fake repair path.

Table 3. Visible control contract after removing manual profile editors and hidden shape sliders. The curated Moana-ocean target is the default shape; visible controls must preserve that shape unless their narrow role explicitly changes scale, morph, steer, position, or display layers.

9 Breaking-wave geometry

The desired default profile is closer to a localized plunging wave than to a cone, cylinder, or arch. The useful approximation is a tapered, asymmetric surface whose highest crest sits behind the curl, whose outer radius is larger than the inner radius, and whose underside remains open. Along the centerline, a breaking wave profile can be represented as three joined curve families:

$$P(u) = \begin{cases} P_{\text{ramp}}(u), & 0 \le u < u_c, \\ P_{\text{crest}}(u), & u_c \le u < u_l, \\ P_{\text{lip}}(u), & u_l \le u \le 1. \end{cases}$$

The ramp rises from the sheet. The crest rounds over. The lip turns forward and downward. The derivative at the terminal lip should point downward when `lipDip` $> 90°$:

$$\left.\frac{dz}{dx}\right|_{\text{tip}} < 0.$$

The final point is allowed to be lower than the farthest horizontal reach. In a real breaking wave, the maximum reach can occur at the shoulder while the lip tip turns down from that shoulder. The downturn must remain open and forward-readable from the side; it must not fold into a closed pucker, dimple, bowl, or side-wall pocket. Treating the farthest-right point as the tip is the mistake that

produces a straight falling chute, while overcorrecting into an under-tucked return is the mistake that produces the rejected cavity. The underside return must also advance back into the flat sheet after the lowest lip sample; a backward floor point is disallowed because it reads as a small lower-lip dimple even when it does not trip a larger bowl metric. The current repair keeps the stronger visible dip by lowering the pendant return and rounding the cap, but bounds the lower branch so the full side render remains a localized plunging sheet rather than a swept tube.

The three-dimensional field must multiply this centerline motion by a span falloff:

$$E(v;u) = \exp\left[-\left(\frac{|v|}{w(u)}\right)^{\alpha}\right],$$

with $w(u)$ varying through the wave. The curl should have a smaller active width than the broad back ramp, so the lip is localized and the open underside remains visible from the side.

10 Mechanism topology

WaveVis uses a rectangular graph of nodes, horizontal profile edges, vertical span edges, and quads as the source lattice. The visible mechanism layer now derives a connected X cell at each lattice node. Each cell renders as a black square body with four visible legs that leave the same center. Opposite leg pairs are equal length and collinear through that center, so the center bisects both bars. Each leg terminates at a spherical connector node that is shared by exactly two adjacent cells. Crossing diagonal families are separated into distinct physical joints so a crossing point cannot become a four-cell connector. The graph is not allowed to become isolated crossed marks, two nearby two-leg pseudo-cells, or a set of isolated rows.

The mechanism evidence from the earlier proof remains relevant as a warning against fake filler surfaces and impossible all-four-equal closure. The current Candidate710 mechanism retains the cell-level X rule Alan selected at Candidate660: each cell has one square center and four straight legs, opposite pairs are collinear and equal length through that center, and each connector is still a spherical physical joint used by exactly two adjacent cells. The old center-pair corridor is now a drift diagnostic, not the hard pass condition (*1*).

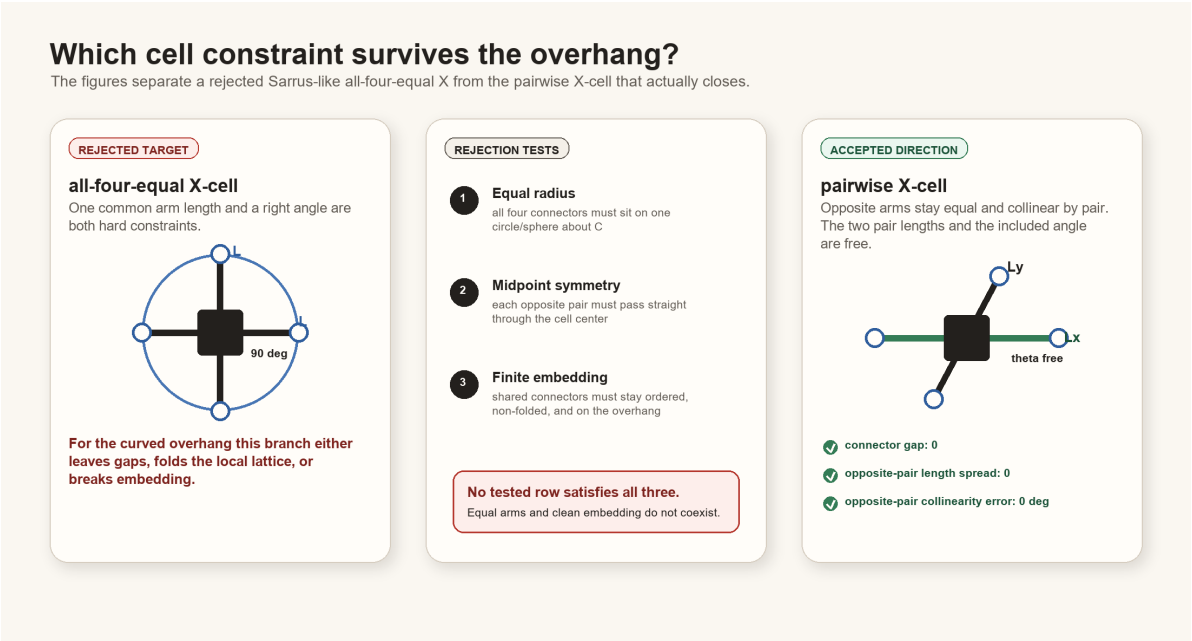

Figure 16. Mechanism contract inherited from the constraint proof and updated in the current renderer. WaveVis should preserve exact shared contact, one-center cells, and two-cell connector joints on adjacent-center spans rather than hiding infeasible geometry behind filler surfaces or a connector-ID count.

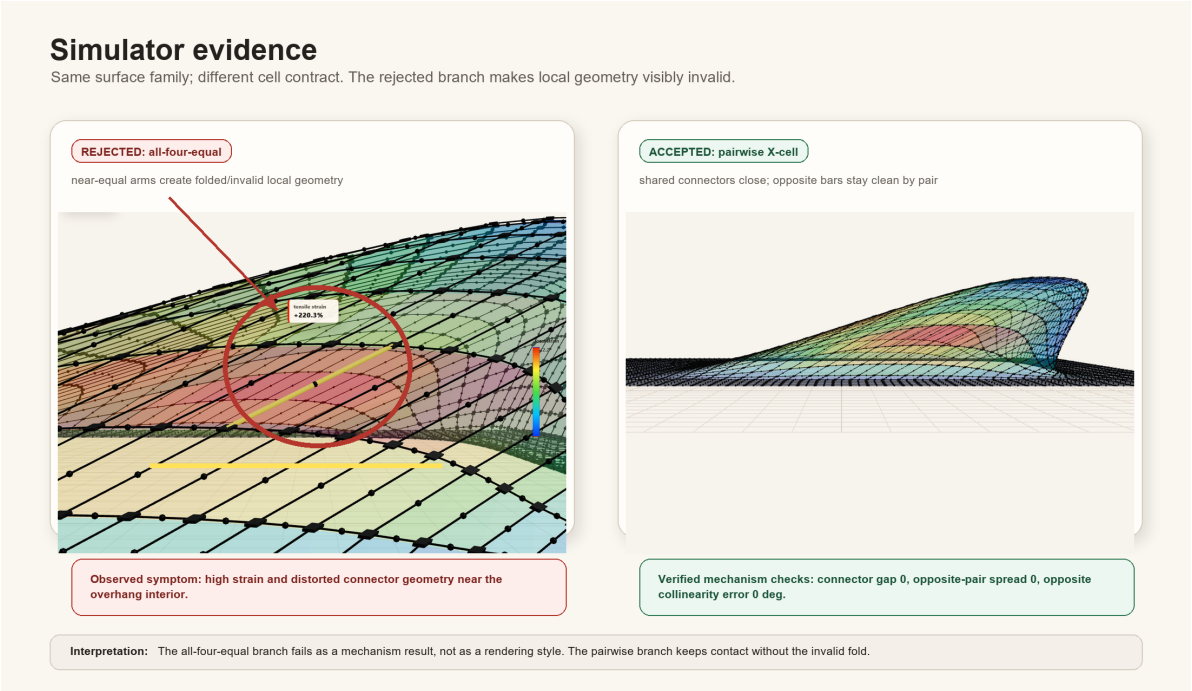

Figure 17. Simulator evidence for the linkage direction. The current architecture must keep this mechanism truth visible while the inverse target shape is improved.

11 Rendering modes

WaveVis has three product views:

- **3D/isometric view** shows the full rectangular array in perspective.
- **Side view** is a camera view of the full three-dimensional render from the side and must show the curl.
- **Top view** shows the full square plan sheet with localized interior deformation and catches terminal caps, triangular fans, wedges, or cropped-patch regressions.

The view contract matters because a side view can hide the curl if lateral geometry forms a wall over the open underside. In that case the correct fix is to reshape the three-dimensional surface so the lateral envelope follows the wave contour and fades out, not to switch the side view into a profile-only debug output or hide the wall.

12 Verification gates

The generator should be checked with both numeric and visual gates. Numeric checks are not sufficient, but they catch repeated hard failures before visual review.

Gate	Measurement	Acceptance intent
Perimeter flatness	$\max_{\partial\Omega} \|p^* - p^0\|$.	Boundary displacement is zero or visually negligible.
Topology completeness	Expected node, edge, and quad counts.	Full rectangular sheet graph remains visible.
Connector closure	Edge endpoints meet their node coordinates.	No detached legs or fake bridges.
Lip curl	Crest height minus lip-nose height; terminal slope; `terminalFaceHasRun`; `terminalSideProfileCavityBlock`.	The lip curls down into a smooth tapered branch rather than ending as a hill, near-vertical chute, hooked tip, lock, dimple, or cavity.
Span localization	Row-wise height profile across y.	Center strong, sides faded; no swept tunnel.
Morph continuity	Residual between adjacent morph samples.	No sudden jumps or topology flips.
Slider isolation	Aligned geometries across position/steer/width changes.	Rigid controls stay rigid; width stays span-only.
Visual side check	Browser side view screenshot or direct render inspection.	Curl visible in the full side view.
Defect repair loop	Confirmed visible defects re-enter source repair.	Acknowledging dimples, pockets, walls, or missing curl is not a stopping state.
`terminalSideProfileCavityBlock`	Machine-readable profile and sampled model side trace.	Before visual QA, reject any lower-branch re-rise, under-lip valley rebound, lower-branch backtrack, abrupt terminal angle jump, or steep visible terminal descent.
Startup URL parser	Covers all public and legacy slider aliases.	A shared verification URL cannot silently load a different height, overhang, lip dip, width, or smoothing state.

Table 4. Verification gates. A build passing TypeScript is not enough; WaveVis must pass the rendered outcome Alan actually uses.

12.1 Current regression command set

The current source-level gate is:

```
npm run check:geometry
```

which expands to:

```
node scripts/check-geometry-invariants.cjs
node scripts/check-inverse-sheet.cjs
```

The inverse-sheet checker compiles the TypeScript geometry into a temporary CommonJS module and then evaluates the default state, low/high lip dip, morph, position, steer, flat contribution, smoothing, width, lateral taper, bowl-pocket, side-render, and startup-URL cases. The geometry-invariant checker covers mechanism-level consistency.

The current acceptance evidence expected from `scripts/check-inverse-sheet.cjs` includes:

- startup display defaults to `showSurface=true`, `showEdges=true`, `showNodes=true`, and `showHeatmap=false`;
- rendered black-square cell count equals the full rectangular grid count 4928;
- rendered center-to-connector X leg segment count equals the expected connector-use count 19400;
- every interior X cell has exactly four legs from one black square body;
- rendered spherical connector-node count equals the expected adjacent connector count 9700;
- each physical connector has exactly two endpoint uses;
- maximum X connector endpoint gap is zero;
- maximum opposite-pair length spread is zero;
- maximum opposite-pair collinearity error is zero degrees;
- maximum opposite-center residual is zero;
- connector corridor drift is bounded as a diagnostic rather than used as the pass condition;
- `sourceUsesCollinearEqualPairConnectors=true`;

- connected X-cell bodies stay close to the sampled wave surface rather than becoming a detached fake mechanism;
- no bowl-pocket count is reported for default, compact-user, and breaking-wide presets;
- the literal readable side trace, startup default, user morph half-lip case, and slider sweeps all pass `terminalSideProfileCavityBlock` before browser screenshots are treated as useful evidence;
- `blockSurfaceBowlPockets` runs on the target grid before morphing and node emission, and the 79-case slider sweep reports no side-cavity or surface-bowl bad cases;
- side-overhang aspect stays in the downturned-lip band after the supplied-line update: startup default aspect between 1.35 and 3.05, compact user morph case between 1.45 and 3.15, and both with reach-to-height between 0.72 and 1.35;
- broad open-curl lip stats keep open-throat, no-dimple, no-backfold, and return-to-flat checks true;
- those lip stats allow a wide rounded cap and tucked nose instead of forcing the old vertical-chute proxy;
- slider robustness reports no bad cases.

These checks are designed around Alan's repeated failures. If a future change removes one of these cases because it is inconvenient, that is a regression unless the PDF and source explain an intentional replacement gate that still catches the same failure.

12.2 Rendered verification

Numeric checks must be followed by rendered verification. The minimum visual pass is:

1. render the exact user URL/preset, not a nearby default;
2. check `Side`: full 3D side camera, visible curl, no profile-only substitution, no wall covering the open underside;
3. check `3D`: full-sheet perspective view, visible curl/overhang, localized center deformation, no swept tunnel, no hidden rows;
4. check `Top`: the sheet should remain square while the interior footprint localizes toward the curl; it must not become a constant extruded strip, broad terminal cap, clipped patch, or large triangular fan;

5. inspect cells/connectors at the curl for zig-zags, random long arms, detached legs, overlaps, or missing four-leg interior connectivity;
6. check lower `lipDip` values as well as 120°, because old low-lip cases became unstable even when max lip looked acceptable;
7. check `morph`=0, 0.5, and 1 because half-morph repeatedly exposed mangled intermediate states.

Screenshots are evidence only when they show the actual rendered canvas and the URL or visible controls identify the tested state. A screenshot of a nice profile line is not evidence that the full linkage array is correct.

13 Build, deploy, and publication runbook

13.1 Local development

Use the WaveVis app root:

`_apps/wavevis.aolabs.io/`.

Typical local commands are:

- `npm run dev` to run the Vite development server;
- `npm run check:geometry` to run geometry and mechanism checks;
- `npm run build` to build the static app;
- `npm run deploy` to publish the app's own GitHub Pages branch.

Do not manually edit `dist`. Build it from source.

13.2 Public fallback deployment

The route Alan can reliably use is `https://aolabs.io/wavevis/`. That route is served from the AO Labs hub/fallback repository, not directly from the WaveVis app repository. Therefore a public WaveVis update requires two publication steps when the fallback route is the user-facing route:

1. build and publish the WaveVis app;
2. copy the built `dist` contents into `_apps/aolabs-site/wavevis/`;

3. commit and push the AO Labs site mirror;
4. verify that `https://aolabs.io/wavevis/` serves the new bundle hash and the rendered artifact.

This is not optional. A source commit or WaveVis `gh-pages` deploy can still leave Alan looking at a stale `aolabs.io/wavevis/` bundle.

13.3 Custom-domain boundary

`https://wavevis.aolabs.io/` is the preferred branded route, but a hostname/certificate failure means it is not the verified artifact. The current Progress source separates `wavevis_home` from `wavevis_custom_domain`. Keep them separate. Do not claim the custom subdomain is fixed until a fresh request verifies the certificate and body from that hostname.

13.4 Progress logging

After meaningful WaveVis work, write a Progress event before final response when practical. The event must include:

- `source_ids`: usually `wavevis_home` and `wavevis_custom_domain`;
- Alan's complaint/request;
- the issue being solved;
- the Codex change;
- the observed public source change;
- provenance: thread, commits, checks, screenshots, and deployment basis;
- uncertainty: exact match not proven, custom domain stale, or any unverified surface.

Progress does not automatically ingest every active Codex chat. If a future WaveVis chat uses this PDF and changes the app, it must still write the event itself.

14 Current verified and open state

This section is intentionally blunt so a future chat does not have to infer whether the system is finished.

Verified current state on June 30, 2026:

- The reliable public route is `https://aolabs.io/wavevis/`.
- The public WaveVis page includes a clickable AO Labs home mark, an app identity mark, the `wavevis.aolabs.io` title, and a `System Architecture PDF` action.
- The Inverse Sheet tab is the first/default option; Double-Layer Sarrus is secondary.
- The current PDF route is the WaveVis fallback paper route: `/wavevis/proofs/`, file `wavevis-system-architecture.pdf`.
- Side, Front, Top, and 3D are the product view modes. Side is the full three-dimensional sheet seen from the side, not a profile-only debug view.
- Current checks pass for the production build, mechanism geometry invariants, and the inverse-sheet regression suite.
- The June 30 Candidate710 checkpoint uses a default surface-plus-X presentation across 3D, Side, Front, and Top, visible black square cell bodies, visible `X legs`, reference-mode mechanism locking, slider robustness, exact collinear equal-pair X-cell invariants, `blockSurfaceBowlPockets` before node emission, a lower side-biased isometric camera, retained 3D-only lower-lip tuck, a smoother no-cavity terminal reference trace, a view-aware readable frame, a Side/3D smoothed curled readable profile, a Side span that avoids both a full-depth wall and a collapsed outline, a separate rounded Top/Front proof profile, a continuous top X overlay without the old terminal split, one-center four-leg X rendering, explicit spherical physical connector nodes, instanced shared-joint rods, a lower-opacity side surface skin, stronger side/3D/front mechanism ink, a denser top grid, a bounded top height-shadow, a perimeter-strip clamp, a terminal taper gate, a 12-column minimum for inspectable X-cell proof routes, and visible-label guards that name `showNodes` as `cells` and `showEdges` as `X legs`.
- The current side render uses a left-to-right side camera with a smooth rise, visible curled throat, downturned terminal branch, connector joints, overhang, an X-only verification path through `surface=0&connectors=1&cells=1`, and a mechanism-locked surface request path through `surface=1&connectors=0&cells=0`. The current 3D render keeps the sheet visible as a gridded surface with connected X cells and avoids the rejected hidden-line, bulb, hard-plateau, lit-oval, opacity-only smoothing branch, overstrong diagonal-crease transform, front-biased flattened

camera, open-C cavity, forward/down vertical-insert profile, busier profile throat-smoothing branch, opposite-camera branches, and the candidate404 side-projection skirt. The side view is less capped than Candidate661, but it remains visibly short of exact June 24 or June 28 reference identity because the curl is still sparse and approximate rather than a full water-like plunging surface. The retained Front render is a spanwise check with pale grid lines, rounded terminal width, controlled cap contours, a front-only pale lit cap cue, a lower projection dome, and visible center X linkage, not a closed reference match. The top render keeps one square reference sheet, one continuous X layer, a stronger reference-sheet wire grid, a guarded mid-section oval projection term, and a height-shadow derived from actual surface height; it does not paste the rejected contour-helper branch into the product view or inherit the side-only profile.

- Browser-rendered side, X-only side, side surface-request-with-mechanism-lock, isometric, front, top, top surface-request-with-mechanism-lock, dense top-X-only, and low-density top-X proof figures were refreshed from Candidate710; a mobile side verification capture was refreshed separately. Current source metrics include 4928 black square cells, 19400 rendered center-to-connector X legs, 19400 rendered shared-joint arms, 9700 spherical physical connector nodes, 9544 checked opposite pairs, interior X-cell leg count range 4–4, zero X connector endpoint gap, zero opposite-pair length spread, zero opposite-pair collinearity error, zero opposite-center residual, exactly two physical endpoint uses for every connector, zero over-occupied physical connectors, sampled readable perimeter `maxEdgeZ` = 0, `maxBorderBandZ` = 0, `maxPlanXOvershoot` = 0, slider robustness 79 checked cases with no bad cases, and source guards requiring the view-aware readable frame, side/isometric smoothed curled readable profile, separate rounded top/front proof profile, one-center/four-leg renderer, collinear equal-pair connector solver, black square cell layer, spherical connector-node layer, visible shared-joint rod layer, mechanism-forward visibility, perimeter-strip fade, top height-shadow branch, hidden axis helpers in X-only proof mode, the 12-column proof range, and the `cells/X legs` visible labels.

Open state that must not be overstated:

- Exact matching to the supplied breaking-wave line drawing, June 24 gridded reference, June 28 stage/cross-section reference, or photorealistic Moana-water frame is not a proven completed result. The correct claim is that the simulator now preserves the hard collinear equal-pair X mechanism and exposes a clearer top-grid footprint while the full visible wave match remains open.

- `https://wavevis.aolabs.io/` is still treated as blocked/stale unless a fresh certificate and body check passes.
- This is a simulator and architecture record. It is not evidence that a physical lattice prototype has been fabricated or experimentally validated.
- If Alan reports a visible dimple, cavity, side wall, bowl, missing curl, zig-zag, disconnected cell, or profile/side confusion after this PDF ships, the live rendered artifact overrides this paper. Repair the source and update the validation gate.

15 Failure modes

The most important failure classes are now explicit:

1. **Swept barrel.** One side profile is dragged sideways at constant span. The result reads as a tunnel, roof, half-pipe, or bridge.
2. **Side-wall cover.** The lip may exist in a center profile, but full side view shows a wall covering the open underside.
3. **Straight chute.** The crest drops nearly vertically to a flat shelf instead of curling forward and under.
4. **Blob or lump.** Excess smoothing or broad support creates a mound with no wave identity.
5. **Top dent.** The outer crest has a kink or concave dent where a smooth radius is required.
6. **Dimple, cavity, or bowl.** The surface creates a pocket, pucker, enclosed hollow, or bowl-like depression rather than a plunging sheet.
7. **Stale URL state.** A browser URL contains height, overhang, width, lipDip, or smoothing values, but startup ignores them and renders a different state.
8. **Zig-zag linkage.** Neighboring legs alternate direction or length erratically.
9. **Hidden mechanism.** Broken topology is ghosted, clipped, or replaced by a cosmetic surface.
10. **Passive defect acknowledgement.** A visible problem is named in chat or recorded in this paper, but the generator and rendered artifact are not repaired and rechecked.

The solution direction is to simplify the target field while strengthening the gates: localized smooth wave first, full array second, mechanism-clean render third. Complexity should be added only if it improves one of those outcomes without breaking the others.

16 Materials and Methods

The implementation source is the WaveVis React and TypeScript app (*2*). The main geometry source is `src/latticeGeometry.ts`. The primary render source is `src/LatticeViewer3D.tsx`. Controls are declared in `src/TargetShapeControls.tsx`. Regression checks are run through `scripts/check-inverse-sheet.cjs`.

The public app serves a static build. The architecture PDF is placed in the public proofs directory as `wavevis-system-architecture.pdf` and linked from the WaveVis control headers. The old connector proof remains a source artifact for the mechanism decision, but the visible paper action now points to this architecture record because the current work is broader than connector assignment.

17 Discussion

The main design lesson is that WaveVis cannot be corrected by tuning only the side profile. The user-facing failure was three-dimensional: side walls, swept tubes, missing full-array linkages, broken side-view semantics, and zig-zag legs. The architecture therefore now uses a single curated target contract rather than a manual editor contract. The sheet generator is responsible for creating a localized Moana-like displacement field with lateral fade, boundary preservation, topology completeness, and a visible curled lip.

This distinction also clarifies future work. If the simulator produces a good center profile but a bad side view, the profile is only evidence that the curve exists somewhere. The real artifact still fails if the full three-dimensional render hides the curl or reads as an extrusion. Similarly, if the mesh is clean but the wave looks like a low mound, the mechanism is not enough. Both the shape and the linkage must be correct.

A newer lesson is that defect naming is not progress by itself. If a rendered state shows dimples, pockets, side walls, or a missing curl, the correct workflow is source repair, geometry-check update, rendered screenshot verification, and public-bundle verification. The current check names this directly as `noVisibleDimplePocket`: the lip must remain an open downturned branch with visible clearance

and a return that advances into the flat sheet instead of folding into a pucker. The same source-of-truth rule applies to URL state. Several correction screenshots used URLs that carried explicit height, overhang, width, lipDip, and smoothing values; ignoring those parameters caused the page to render a different geometry than the requested test state. The startup parser is now part of validation through `startupUrlParamCoverage`. The architecture record is useful only if it changes the generator, state loader, and acceptance loop; it cannot substitute for a corrected human-facing WaveVis render.

18 Acknowledgements

This record was written to reduce repeated handoff burden during WaveVis development. The repeated complaints about side walls, missing cells, zig-zag legs, profile-view confusion, and swept-barrel geometry are treated as acceptance criteria rather than optional polish notes.

19 Funding

No external funding is claimed in this system architecture record.

20 Author contributions

Alan N. Pham defined the target artifact, accepted and rejected visual states, and the mechanism constraints. Codex generated this architecture record from the current WaveVis source and the active development contract.

21 Competing interests

The author declares no competing interests for this internal architecture record.

22 Data availability

The source data for this record is the WaveVis source tree, the Alan-supplied Moana ocean reference images copied into the proof figure set, the local proof diagrams, and the current simulator behavior. The public PDF is served with the WaveVis static app.

23 Code availability

The relevant implementation files are `src/latticeGeometry.ts`, `src/LatticeViewer3D.tsx`, `src/TargetShapeControls.tsx`, and `scripts/check-inverse-sheet.cjs` in the WaveVis app repository.

24 Additional information

This document supersedes the old header link to the narrow connector-contact constraint proof as the primary public WaveVis PDF. The connector proof remains an implementation reference for mechanism feasibility, but the current public artifact needs the broader system architecture and outcome-verification contract.

25 References

1. A. N. Pham, *WaveVis overhang connector assignment: evidence rejecting all-four-equal X-cells*, Internal WaveVis constraint proof and simulator evidence figures, 2026.
2. A. N. Pham, *WaveVis inverse-sheet simulator source tree*, Local source files: src/latticeGeometry.ts, src/LatticeViewer3D.tsx, src/TargetShapeControls.tsx, scripts/check-inverse-sheet.cjs, 2026.